

Axhenti Alessandro, 110671

AA 2026

Project 3

MIPS

MIPS Assembly Language

**PROGETTO
CALCOLATORI
ELETTRONICI**

Indice

Introduzione.....	2
Logica del gioco e dati.....	3
Flow chart.....	4
Funzioni cruciali.....	5
1) Creazione dell'ArrayList.....	6.1
2) Inserimento di un elemento.....	6.2
3) Ricerca elemento.....	6.3
4) Classifica Giocatori.....	7.1
5) Cerca In roster ricorsivo.....	7.2
Popolamento DB e test.....	8
Altre Informazioni.....	8

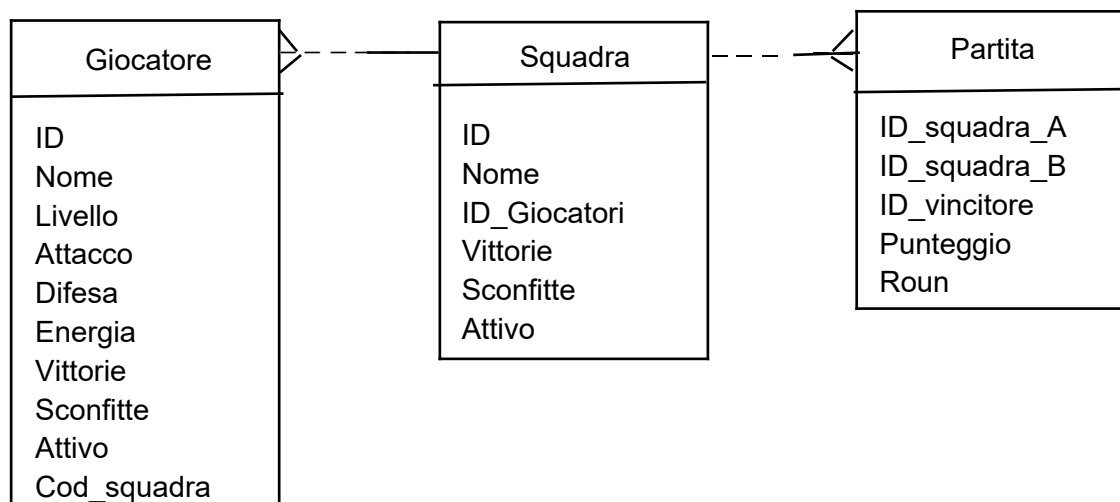
Introduzione

Il presente elaborato descrive lo sviluppo del Project 3 – Turn-Based Strategy Tournament Manager, realizzato in linguaggio assembler MIPS. L'obiettivo del progetto è la realizzazione di un sistema per la gestione di un torneo a turni, in grado di amministrare giocatori, squadre e partite attraverso un'interfaccia basata su menu. Il programma consente di inserire, ricercare e visualizzare i dati, simulare duelli tra giocatori e incontri tra squadre, registrare i risultati delle partite, generare classifiche e statistiche e gestire l'eliminazione logica dei record. Dal punto di vista implementativo, il progetto è stato sviluppato adottando una struttura modulare, suddividendo il codice in più file sorgente, ciascuno dedicato a una specifica funzionalità. Per la memorizzazione dei dati è stata realizzata una struttura dinamica di tipo ArrayList, utilizzata per gestire collezioni di giocatori, squadre e partite. Tale struttura permette l'inserimento dinamico di nuovi elementi e l'espansione automatica della memoria disponibile quando la capacità iniziale viene raggiunta. L'applicazione è organizzata attorno a un menu principale che permette all'utente di accedere a tutte le funzionalità previste dalla traccia del progetto. La selezione delle operazioni avviene mediante una jump table, che consente di richiamare direttamente la routine associata alla scelta effettuata, migliorando l'organizzazione e la leggibilità del codice. Nei capitoli successivi vengono illustrati il diagramma di flusso del programma, le principali strutture dati adottate, le porzioni di codice più significative con la relativa spiegazione e le modalità con cui è stato popolato il database iniziale, così da consentire la verifica di tutte le funzionalità richieste dalla consegna.

Logica del gioco

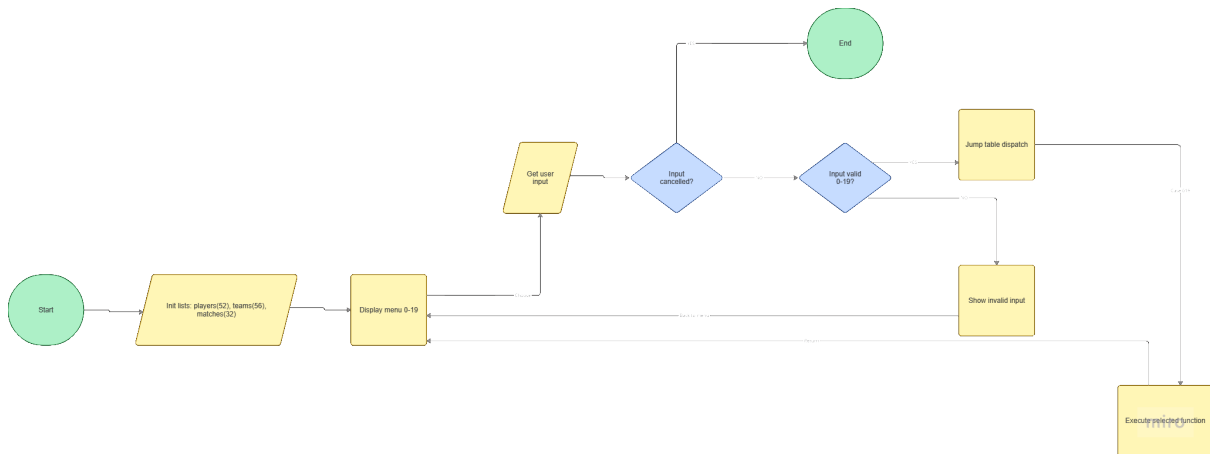
Il programma simula un torneo tra squadre composte da un massimo di cinque giocatori. Ogni giocatore possiede caratteristiche proprie, quali livello, punti attacco, punti difesa ed energia, che influenzano l'esito dei combattimenti. Le squadre vengono create associando ai rispettivi roster i giocatori precedentemente registrati nel sistema. Durante una simulazione è possibile effettuare un duello tra due giocatori o una partita tra due squadre. Nel duello, i giocatori si attaccano a turni fino all'esaurimento dell'energia di uno dei due; il vincitore incrementa il numero di vittorie, mentre il perdente incrementa il numero di sconfitte. Nella partita tra squadre vengono invece confrontati i giocatori dei due roster attraverso una serie di duelli, determinando la squadra vincitrice, che viene registrata nello storico delle partite insieme al numero di round disputati. Sulla base dei risultati ottenuti, il sistema aggiorna classifiche e statistiche, consentendo di individuare il giocatore e la squadra più forti.

Struttura dei dati

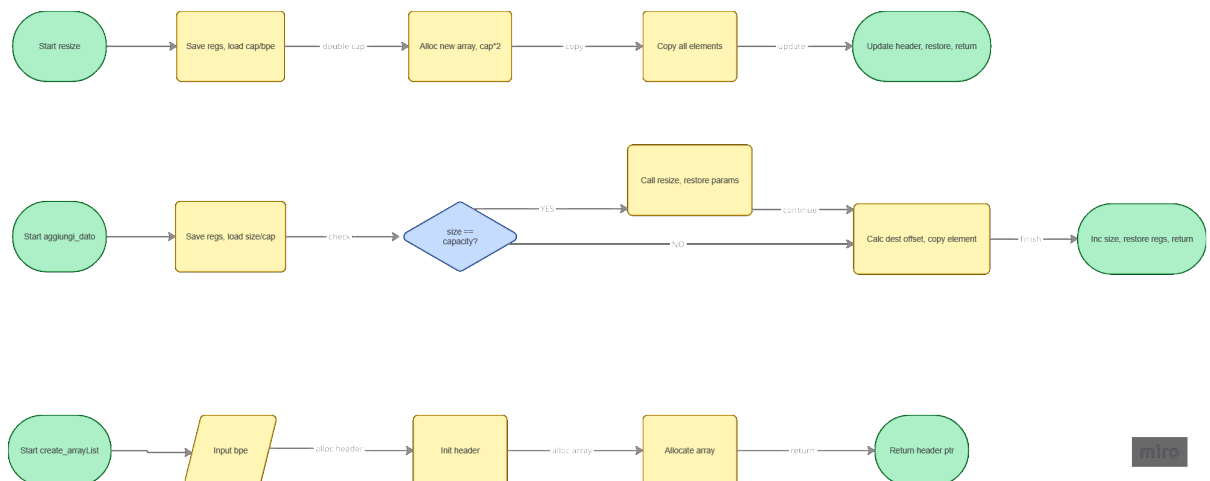


Flow Charts

Main:

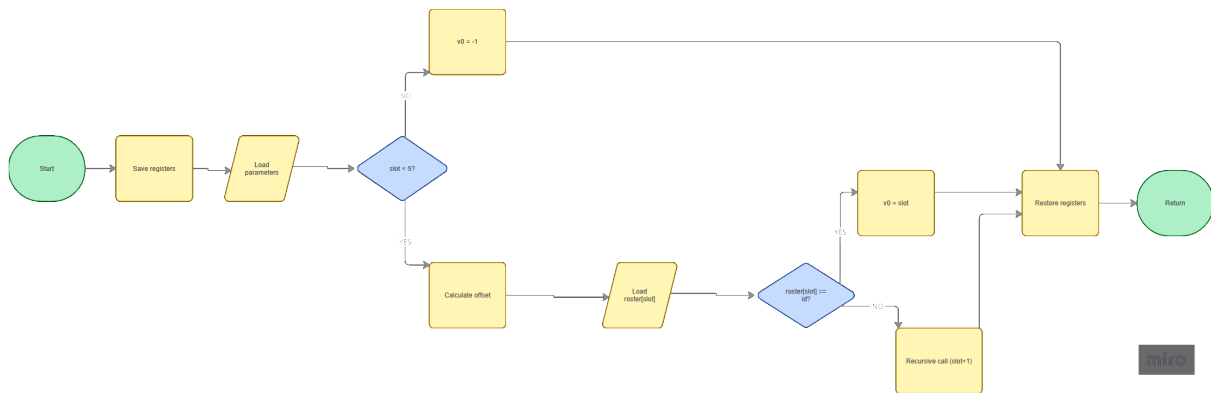


Funzioni Gestione Dati e ArrayList:

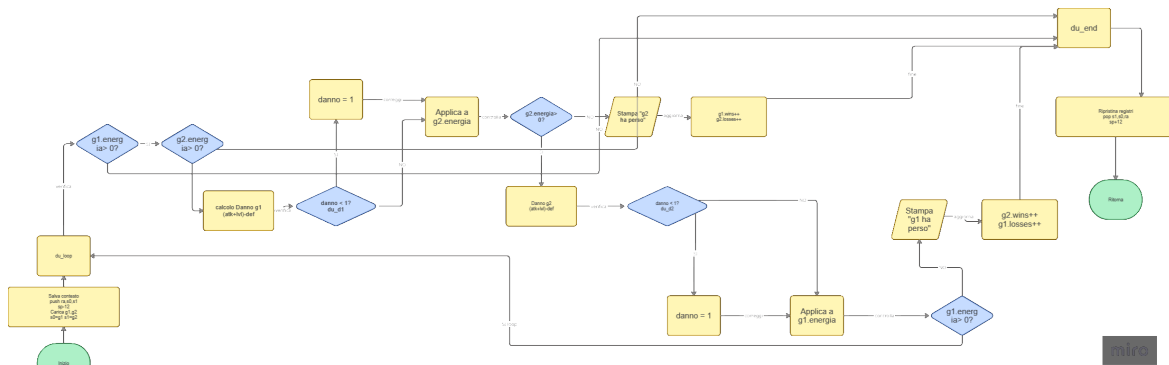


miro

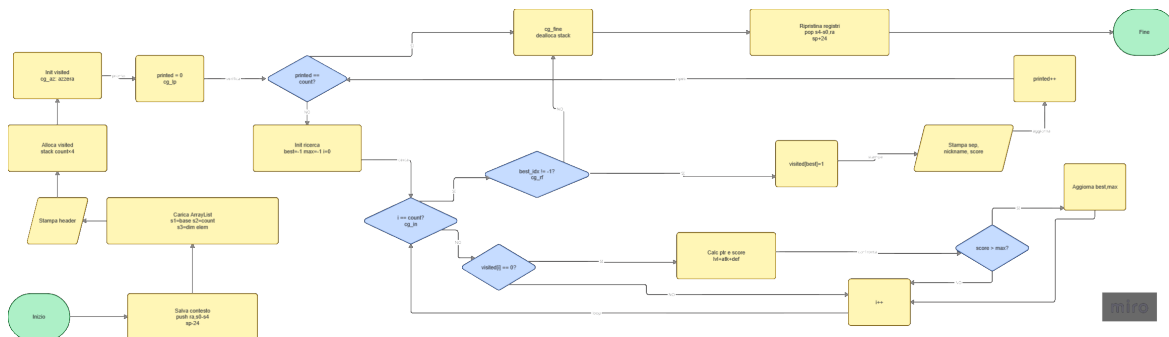
Funzione roster ricorsivo



Duello:



Giocatore Migliore:



Flusso Logico Delle Funzioni Cruciali:

Creazione dell'ArrayList (create_arrayList)

La funzione realizza la struttura dati dinamica utilizzata per memorizzare giocatori, squadre e partite.

- 1) Alloca in memoria l'header dell'ArrayList.
- 2) Inizializza i campi size, capacity e dimensione elemento.
- 3) Alloca il blocco di memoria destinato agli elementi.
- 4) Memorizza nell'header il puntatore al blocco dati.
- 5) Restituisce il puntatore all'ArrayList.

L'ArrayList è composta da un header contenente il puntatore all'array, il numero di elementi presenti, la capacità massima e la dimensione di ciascun elemento. Questa struttura consente di gestire dinamicamente la memoria senza utilizzare array statici di grandi dimensioni.

Inserimento di un elemento (aggiungi_dato)

La funzione inserisce un nuovo elemento all'interno dell'ArrayList.

- 1) Salva dati sullo stack.
- 2) Carica il numero di elementi e la capacità dell'ArrayList.
- 3) Verifica se l'array è pieno.
- 4) Se necessario richiama la funzione resize per espandere la memoria.
- 5) Calcola l'indirizzo della prima posizione libera.
- 6) Copia l'intero record nella nuova posizione, una word (4 byte) alla volta.
- 7) Incrementa il numero di elementi presenti (size).
- 8) Ripristina i dati e ritorna al chiamante.

La copia dell'elemento viene effettuata tramite un ciclo che simula il comportamento della funzione memcpy, copiando il record dalla memoria sorgente alla memoria destinazione.

Ricerca di un elemento (cerca_ptr)

La funzione ricerca un elemento all'interno dell'ArrayList utilizzando il relativo identificativo.

- 1) Carica il puntatore all'array e il numero di elementi.
- 2) Inizia una scansione lineare dell'ArrayList.
- 3) Per ogni elemento legge il campo ID.
- 4) Confronta l'ID con quello richiesto.
- 5) Se trova una corrispondenza restituisce il puntatore al record.
- 6) Se termina la scansione senza trovare l'elemento restituisce valore nullo.

L'algoritmo utilizzato è una ricerca lineare, con complessità **O(n)**.

Classifica dei giocatori (stampa_classifica_giocatori)

La funzione stampa la classifica dei giocatori ordinandoli in ordine decrescente in base a un punteggio calcolato come somma di livello, attacco e difesa.

- 1) Salva il contesto della funzione e carica dall'ArrayList il puntatore ai giocatori, il numero di elementi e la dimensione di ciascun record.
- 2) Stampa il titolo della classifica.
- 3) Alloca sullo stack un vettore visitati, inizializzato a zero, utilizzato per ricordare i giocatori già stampati.
- 4) Per ogni posizione della classifica esegue una scansione completa dell'ArrayList.
- 5) Ignora i giocatori già visitati.
- 6) Calcola il punteggio di ciascun giocatore sommando livello, attacco e difesa.
- 7) Individua il giocatore con il punteggio massimo.
- 8) Marca tale giocatore come visitato.
- 9) Stampa nickname e punteggio.
- 10) Ripete la procedura fino a stampare tutti i giocatori.
- 11) Libera la memoria temporanea utilizzata sullo stack e ripristina il contesto.

L'algoritmo implementa una variante del Selection Sort, mantenendo invariato l'ordine originale dei record presenti nell'ArrayList.

Ricerca ricorsiva nel roster (cerca_in_roster_ricorsivo)

Questa funzione implementa la funzionalità ricorsiva richiesta dalla traccia del progetto, effettuando la ricerca di un giocatore all'interno del roster di una squadra.

- 1) Salva il contesto della funzione sullo stack.
- 2) Riceve come parametri il puntatore alla squadra, l'ID del giocatore da cercare e lo slot corrente del roster.
- 3) Verifica il caso base della ricorsione:
- 4) se lo slot corrente è maggiore o uguale a cinque, la ricerca termina restituendo -1.
- 5)
- 6) Calcola l'indirizzo dello slot corrente del roster.
- 7) Legge l'ID del giocatore presente nello slot.
- 8) Se l'ID coincide con quello cercato restituisce l'indice dello slot.
- 9) In caso contrario richiama ricorsivamente la funzione incrementando lo slot corrente.
- 10) Al termine della chiamata ricorsiva ripristina il contesto e restituisce il risultato al chiamante.

La funzione realizza una ricerca lineare ricorsiva. Ogni chiamata analizza uno slot del roster e delega la ricerca dello slot successivo alla chiamata ricorsiva, fino al raggiungimento del caso base o al ritrovamento del giocatore. La complessità dell'algoritmo è $O(n)$, con $n=5n = 5n=5$, corrispondente al numero massimo di giocatori presenti in una squadra.

Popolamento del database e test

All'avvio del programma è disponibile l'opzione 0 – Popolamento del database, che inizializza automaticamente tutte le strutture dati necessarie al funzionamento dell'applicazione. Il popolamento viene effettuato tramite l'inserimento di record predefiniti, in modo da consentire l'esecuzione e la verifica di tutte le funzionalità richieste dalla traccia senza dover inserire manualmente tutti i dati.

In particolare vengono creati:

- 20 giocatori, ciascuno con un ID univoco, nickname, livello, punti attacco, punti difesa, energia iniziale, numero di vittorie, numero di sconfitte e flag di attivazione.
- 20 squadre, ciascuna identificata da un ID e da un nome. Alcune squadre risultano già complete con il proprio roster, mentre altre sono lasciate parzialmente vuote per permettere di testare l'assegnazione dei giocatori.
- 20 partite, contenenti l'identificativo della partita, gli ID delle due squadre coinvolte, la squadra vincitrice, il numero di round disputati e il flag di attivazione.

Il popolamento iniziale è stato progettato in modo da rendere immediatamente testabili tutte le funzionalità previste dal progetto

- L'Aggiunta del 21-esimo/a giocatore/squadra (per verificare funzione resize)
- la ricerca di giocatori tramite ID;
- la ricerca di squadre tramite nome;
- l'assegnazione di nuovi giocatori alle squadre;
- la simulazione di duelli tra giocatori;
- la simulazione di partite tra squadre;
- la registrazione e la visualizzazione dello storico delle partite;
- la generazione della classifica dei giocatori e delle squadre;
- l'individuazione del giocatore e della squadra più forti;
- la stampa dei giocatori non assegnati ad alcuna squadra;
- l'eliminazione logica di giocatori e squadre;
- la funzionalità ricorsiva di ricerca di un giocatore all'interno del roster di una squadra.

Per eseguire correttamente i test è sufficiente selezionare inizialmente l'opzione 0 del menu.

Da questo momento tutte le altre funzionalità possono essere utilizzate direttamente, senza la necessità di inserire manualmente i dati.

Altre informazioni utili

Durante lo sviluppo del progetto è stato fatto uso di strumenti di intelligenza artificiale per supporto alla realizzazione di alcune parti del codice e della documentazione. In particolare, l'AI è stata utilizzata per implementare le funzionalità [16:19], per migliorare la formattazione e l'aspetto dell'output del programma, per la stesura e l'organizzazione dei commenti nel codice e per la redazione della documentazione tecnica complessiva.